Project Pilot 作品介绍

Project Pilot 是一个面向软件想法的 AI 项目工作台。用户用一句话描述想做的东西，系统把想法拆成边界清楚的任务，为每项任务推荐当前可用的模型，在隔离目录里实现，再集成并通过自动验证。整个过程可以在界面上看到：谁在做、写出了什么、检查有没有通过、哪里需要人来决定。

1. 项目背景

把一个软件想法交给多个 AI 一起做，常见做法是先搭一套“公司”：部门、员工、汇报关系和长期在线的 Agent。对只想做成一个具体项目的人来说，这套结构门槛太高。使用者往往要先理解组织、适配器、密钥和权限，还分不清哪一次运行改了哪些文件。

Project Pilot 把问题收窄成一次项目交付：

- 默认使用者不熟悉技术栈、模型和 API。
- 不要求先注册密钥。密钥只在真正调用外部模型时才需要。
- 不建立长期部门和员工档案。规划、界面、逻辑、素材、测试和集成都是这次项目里的任务，做完就结束。
- 每次运行、每次文件变更和每次验证都留在项目里，可以回看。

产品因此分成两条线。一条是控制：拆任务、推荐模型、确认方案、在失败时请用户选择。另一条是执行：真正写文件、调用模型、跑验证。模型可以更换，执行器保持同一个。

2. 技术架构

系统在用户自己的电脑上运行。页面由本机服务直接打开，项目数据也保存在本机，不依赖外部数据库。

```
浏览器界面
  左侧：New Chat / Mine / Projects
  中间：想法输入、任务树、工作进程
  右侧：工作进程 / 需要你处理 / 最近变化
        |
        |  HTTP + Server-Sent Events
        v
Node 服务
  规划：把想法拆成任务和接口契约
  推荐：按偏好、地区、预算和已有连接选择模型
  执行：隔离目录、并行波次、集成、验证
  存储：JSON 文件；API Key 只保存密文
```

模块	职责

web/	深色三栏界面。新建项目只有一个输入框，技术路线由系统建议后点选。
src/planner	识别游戏或一般软件，生成规划、架构、实现、集成和验证任务，并写明依赖。
src/router	先做硬过滤，再按能力、偏好、历史、成本和稳定性打分。
src/executor	自有执行器。本地试用不调用外部模型；已连接的 OpenAI 兼容接口（如 DeepSeek）按统一工具协议写文件。
src/service	项目、确认、执行、暂停、终止和需要用户处理的决定。
src/store	本地 JSON。密钥使用 AES-256-GCM 加密，界面只显示后四位。

一次项目的数据包括：任务、运行记录、变更、验证、决定和产物。任务状态从待执行到进行中、待确认、已完成；失败时进入需要用户处理，并给出重试、缩小范围、更换模型或交给用户。

执行时，互不依赖的任务放在同一波次、各自的目录里。集成任务再把依赖产物收拢，验证任务运行 `node verify.mjs`。确认方案时可以选择本机上的一个文件夹，后续文件写进这个文件夹；没有选择时，写在应用的临时工作区。

模型偏好可以随时修改，覆盖顺序是：单个任务，然后是项目，最后是账户默认。预设包括平衡、质量优先、性价比优先、速度优先、只用国内服务和更注意隐私。推荐结果给出主推荐、备选和原因，不把某个模型说成永远最强。

3. 应用场景

- **把一句话做成可打开的小软件。**例如“做一个能移动和跳跃的网页小游戏”。系统拆出画面、逻辑、关卡、集成和验证，并给出一个可以在浏览器打开的页面。
- **按约束组合多个模型。**国内服务、预算或隐私要求不同的任务，可以使用不同模型。画面和逻辑可以并行，再进入同一次集成。
- **先看方案，再决定是否花钱。**没有 API Key 时仍能生成蓝图和任务树，并用本地试用执行器跑通流程。接入 DeepSeek 等 OpenAI 兼容服务后，同一套执行器改为调用该服务。
- **查看一次失败停在哪里。**验证没通过或模型没有写出文件时，项目停在“需要你处理”，并保留运行日志和变更记录。

`npm test` 覆盖偏好覆盖、国内模型过滤、并行拆分、密钥加密和端到端执行。

4. 创新点

1. **以项目为中心，而不是以 Agent 公司为中心。**用户描述想法就能开始，不需要先创建部门、员工或长期工作流。
2. **任务按能力临时组合。**规划、架构、界面、逻辑、素材、测试和集成是这次交付的角色，不是永久员工。复杂工作时，不同产物可以并行，再按契约集成。

3. **拆开之后，为每一部分推荐最合适的模型，把不同模型的长处合在一起。**系统不为整个项目只指定一个模型。规划、界面、逻辑、画面、测试各自得到一份推荐：推理和方案偏向更强的推理模型，常规编写偏向更快或更省的模型，画面则使用能生成图像的模型。这些部分可以并行完成，再集成到同一次交付里。
4. **推荐会跟着用户的回答变。**用户如果说用不了国外的 AI，推荐就只留在国内服务里；如果说更看重性价比、质量或速度，排序就会跟着改。这些选择之后还能改，也可以只对一个项目或一项任务生效。
5. **只在用户已经保存的密钥里做选择。**事先存入的 API Key 会参与推荐，已经能调用的模型优先。没有密钥时仍然可以先看任务拆分和方案；接入密钥之后，同一套推荐改为使用这些已保存的服务。
6. **门槛放在默认路径上。**新建只有一个对话框。技术栈不要求用户先填写，而是给出可点选的建议。
7. **过程可以回看。**每一次运行都留下日志、改过的文件和验证结果。失败时停下来，请用户选择重试、缩小范围、更换模型或自己处理。

5. 如何运行

环境：Node.js 24。

```
npm install
npm test
npm run dev
```

浏览器打开 http://127.0.0.1:3210 。在 New Chat 输入想法并选择建议的技术路线，确认方案后点击开始实现。本地试用不需要 API Key。要调用 DeepSeek 时，在 Mine 中保存服务地址和密钥。

本仓库不包含本机数据目录。API Key 和本地数据库不会进入版本库。